? Федеральное государственное автономное образовательное
учреждение высшего образования
Национальный исследовательский университет ИТМО

Факультет программной инженерии и компьютерной техники
Направление подготовки 09.03.04 Программная инженерия
Дисциплина «*Сервис-ориентированная архитектура*»

Отчёт
По лабораторной работе №2

Вариант: 21

Сдающие:
Петров Вячеслав Маркович
Елисеев Константин Иванович

группа P3408

Преподаватель:
Кривоносов Егор Дмитриевич

г. Санкт-Петербург, 2026 г.

Задание

На основе разработанной в рамках лабораторной работы #1 спецификации реализовать два веб-сервиса и использующее их API клиентское приложение.

Требования к реализации и развёртыванию сервисов:

- Первый ("вызываемый") веб-сервис должен быть реализован на фреймворке Spring MVC REST и развёрнут в окружении под управлением сервера приложений Tomcat.
- Второй веб-сервис должен быть реализован на фреймворке Spring MVC REST, развёрнут в окружении под управлением ещё одного экземпляра сервера приложений Tomcat и вызывать REST API первого сервиса.
- **Для обоих сервисов необходимо реализовать все функции, задокументированные в API, в строгом соответствии со спецификацией!**
- Доступ к обоим сервисам должен быть реализован по протоколу https с самоподписанным сертификатом сервера. Доступ к сервисам посредством http без шифрования должен быть запрещён.

Требования к клиентскому приложению:

- Клиентское приложение может быть написано на любом веб-фреймворке, который можно запустить на сервере helios.
- Приложение должно обеспечить полный набор возможностей, предоставляемых API обоих сервисов - включая сортировку, фильтрацию и постраничный вывод элементов коллекции.
- Приложение должно преобразовывать передаваемые сервисами данные в человеко-читаемый вид - параграф текста, таблицу и т.д.
- Клиентское приложение должно информировать пользователя об ошибках, возникающих на стороне сервисов, в частности, о том, что сервису были отправлены невалидные данные.

Оба веб-сервиса и клиентское приложение должны быть развёрнуты на сервере helios.

Выполнение

Исходный код

https://github.com/virdxk/ticket-api-openapi

- https://github.com/virdxk/ticket-api-openapi/tree/main/ticket-service - Ticket Service
- https://github.com/virdxk/ticket-api-openapi/tree/main/person-service - Person Service
- https://github.com/virdxk/ticket-api-openapi/tree/main/booking-service - Booking Service
- https://github.com/virdxk/ticket-api-openapi/tree/main/common - общий модуль: формат ошибок, фильтрация, сортировка, пагинация
- https://github.com/virdxk/ticket-api-openapi/tree/main/client - клиентское приложение
- https://github.com/virdxk/ticket-api-openapi/tree/main/deploy - скрипты развёртывания на helios

Реализация сервисов

Все три сервиса (Ticket, Person и Booking) реализованы на Spring Boot 3.5 (Spring MVC REST) и собираются в WAR-архивы. Интерфейсы контроллеров и модели сгенерированы из YAML-спецификаций плагином openapi-generator-maven-plugin, поэтому API совпадает со спецификацией. Booking Service вызывает Ticket Service и Person Service через клиенты, сгенерированные из тех же спецификаций.

Код каждого сервиса разложен по пакетам controller, service, repository и entity. Общий код вынесен в модуль common:

- единый формат ошибки {message, time, details}, тексты ошибок хранятся в messages.properties
- фильтрация, сортировка и постраничная выборка
- проверка JSON запроса: все нарушения возвращаются одним ответом 422 со списком в details

Развёртывание

Каждый сервис развёрнут в отдельном экземпляре Tomcat 10.1.60 на helios.

Порты Tomcat закрыты снаружи, поэтому запросы идут через se.ifmo.ru. Для каждого сервиса в public_html есть каталог с .htaccess и скриптом, который пересылает запрос в Tomcat по https и возвращает ответ. Клиент лежит в public_html/lab2.

Компонент	Порт Tomcat	Публичный адрес
Ticket Service	41390	https://se.ifmo.ru/~s412939/ticket-service/tickets
Person Service	41391	https://se.ifmo.ru/~s412939/person-service/persons
Booking Service	41392	https://se.ifmo.ru/~s412939/booking-service/booking
Клиент	-	https://se.ifmo.ru/~s412939/lab2/
Swagger UI	-	https://se.ifmo.ru/~s412939/

Доступ по https

Самоподписанный сертификат создаёт утилитой keytool скрипт развёртывания при первом запуске на helios: хранилище ключа keystore.p12, хранилище доверенных сертификатов truststore.p12 и сертификат server.crt. Пути к ним и пароли указаны в server.xml Tomcat и в application.properties Booking Service. В server.xml есть только https-коннектор, поэтому запросы по http отклоняются.

```
$ curl http://localhost:41390/tickets
Bad Request
This combination of host and port requires TLS.



$ curl https://localhost:41390/tickets
curl: (60) SSL certificate problem: self-signed certificate

$ curl --cacert ~/public_html/ticket-service/server.crt -o /dev/null -w "%{http_code}"
https://localhost:41390/tickets
200

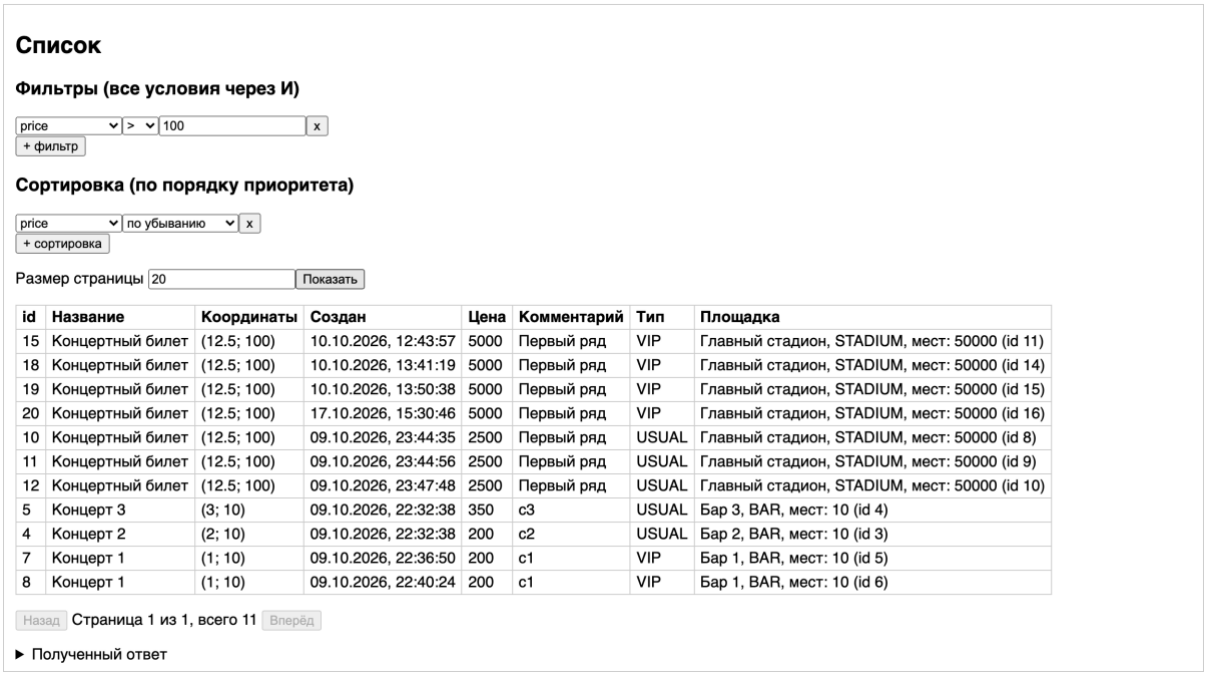
$ openssl x509 -in ~/public_html/ticket-service/server.crt -noout -subject -issuer -
ext subjectAltName
subject=CN = helios.cs.ifmo.ru
issuer=CN = helios.cs.ifmo.ru
X509v3 Subject Alternative Name:
    DNS:localhost, DNS:helios.cs.ifmo.ru, IP Address:127.0.0.1
```

Клиентское приложение

https://se.ifmo.ru/~s412939/lab2/

Клиент написан на React. Для каждой коллекции есть список с фильтрацией по любому полю, многоуровневой сортировкой и постраничным выводом, получение, создание, частичное обновление и удаление элемента, а также

специальные операции. Данные выводятся таблицами. При ошибке показываются код ответа, сообщение и список нарушений по полям.



Список

Фильтры (все условия через И)

price > 100 x

+ фильтр

Сортировка (по порядку приоритета)

price по убыванию x

+ сортировка

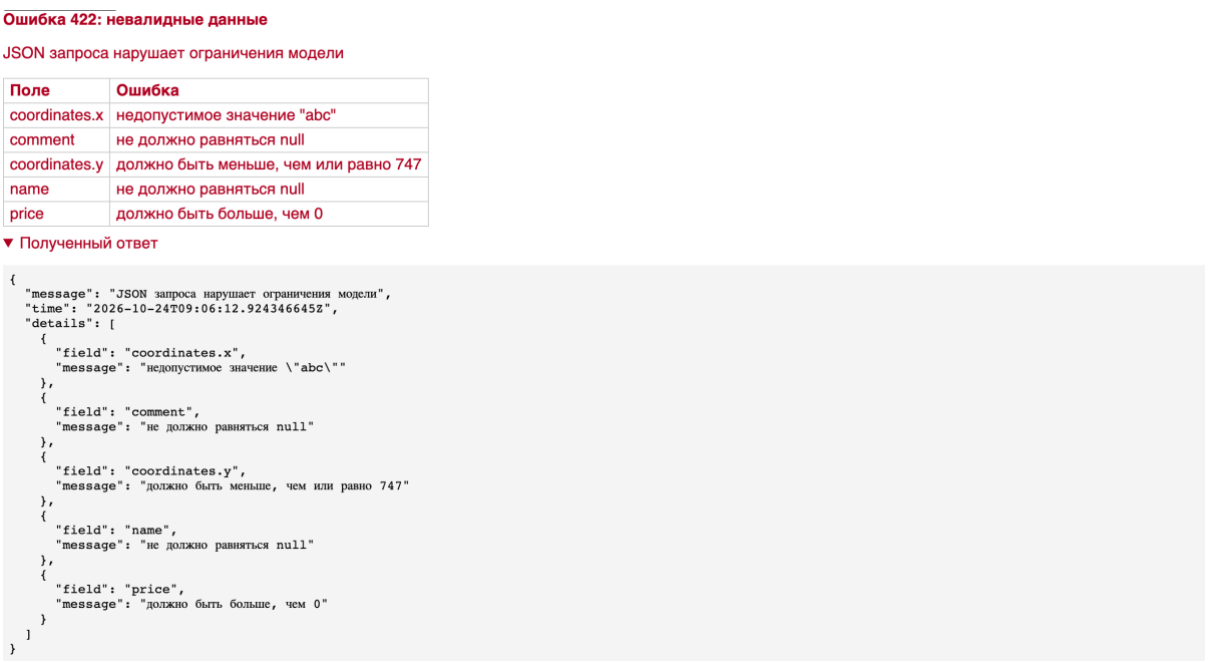
Размер страницы 20 Показать

id	Название	Координаты	Создан	Цена	Комментарий	Тип	Площадка
15	Концертный билет	(12.5; 100)	10.10.2026, 12:43:57	5000	Первый ряд	VIP	Главный стадион, STADIUM, мест: 50000 (id 11)
18	Концертный билет	(12.5; 100)	10.10.2026, 13:41:19	5000	Первый ряд	VIP	Главный стадион, STADIUM, мест: 50000 (id 14)
19	Концертный билет	(12.5; 100)	10.10.2026, 13:50:38	5000	Первый ряд	VIP	Главный стадион, STADIUM, мест: 50000 (id 15)
20	Концертный билет	(12.5; 100)	17.10.2026, 15:30:46	5000	Первый ряд	VIP	Главный стадион, STADIUM, мест: 50000 (id 16)
10	Концертный билет	(12.5; 100)	09.10.2026, 23:44:35	2500	Первый ряд	USUAL	Главный стадион, STADIUM, мест: 50000 (id 8)
11	Концертный билет	(12.5; 100)	09.10.2026, 23:44:56	2500	Первый ряд	USUAL	Главный стадион, STADIUM, мест: 50000 (id 9)
12	Концертный билет	(12.5; 100)	09.10.2026, 23:47:48	2500	Первый ряд	USUAL	Главный стадион, STADIUM, мест: 50000 (id 10)
5	Концерт 3	(3; 10)	09.10.2026, 22:32:38	350	c3	USUAL	Бар 3, BAR, мест: 10 (id 4)
4	Концерт 2	(2; 10)	09.10.2026, 22:32:38	200	c2	USUAL	Бар 2, BAR, мест: 10 (id 3)
7	Концерт 1	(1; 10)	09.10.2026, 22:36:50	200	c1	VIP	Бар 1, BAR, мест: 10 (id 5)
8	Концерт 1	(1; 10)	09.10.2026, 22:40:24	200	c1	VIP	Бар 1, BAR, мест: 10 (id 6)

Назад Страница 1 из 1, всего 11 Вперёд

▸ Полученный ответ

Рис 1. Список билетов с фильтрацией, сортировкой и постраничным выводом



Ошибка 422: невалидные данные

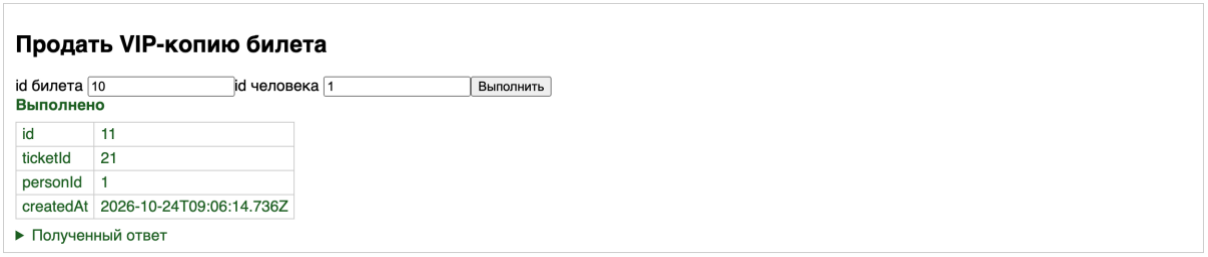
JSON запроса нарушает ограничения модели

Поле	Ошибка
coordinates.x	недопустимое значение "abc"
comment	не должно равняться null
coordinates.y	должно быть меньше, чем или равно 747
name	не должно равняться null
price	должно быть больше, чем 0

▾ Полученный ответ


```
{
  "message": "JSON запроса нарушает ограничения модели",
  "time": "2026-10-24T09:06:12.924346645Z",
  "details": [
    {
      "field": "coordinates.x",
      "message": "недопустимое значение \"abc\""
    },
    {
      "field": "comment",
      "message": "не должно равняться null"
    },
    {
      "field": "coordinates.y",
      "message": "должно быть меньше, чем или равно 747"
    },
    {
      "field": "name",
      "message": "не должно равняться null"
    },
    {
      "field": "price",
      "message": "должно быть больше, чем 0"
    }
  ]
}
```


Рис 2. Ответ 422 на создание билета с невалидными данными



Продать VIP-копию билета

id билета 10 id человека 1 Выполнить

Выполнено

id	11
ticketId	21
personId	1
createdAt	2026-10-24T09:06:14.736Z

▸ Полученный ответ

Рис 3. Продажа VIP-копии билета через Booking Service



Рис 4. Список бронирований

Swagger UI

https://se.ifmo.ru/~s412939/

В спецификациях указаны публичные https-адреса сервисов, поэтому операции можно выполнять прямо из Swagger UI.

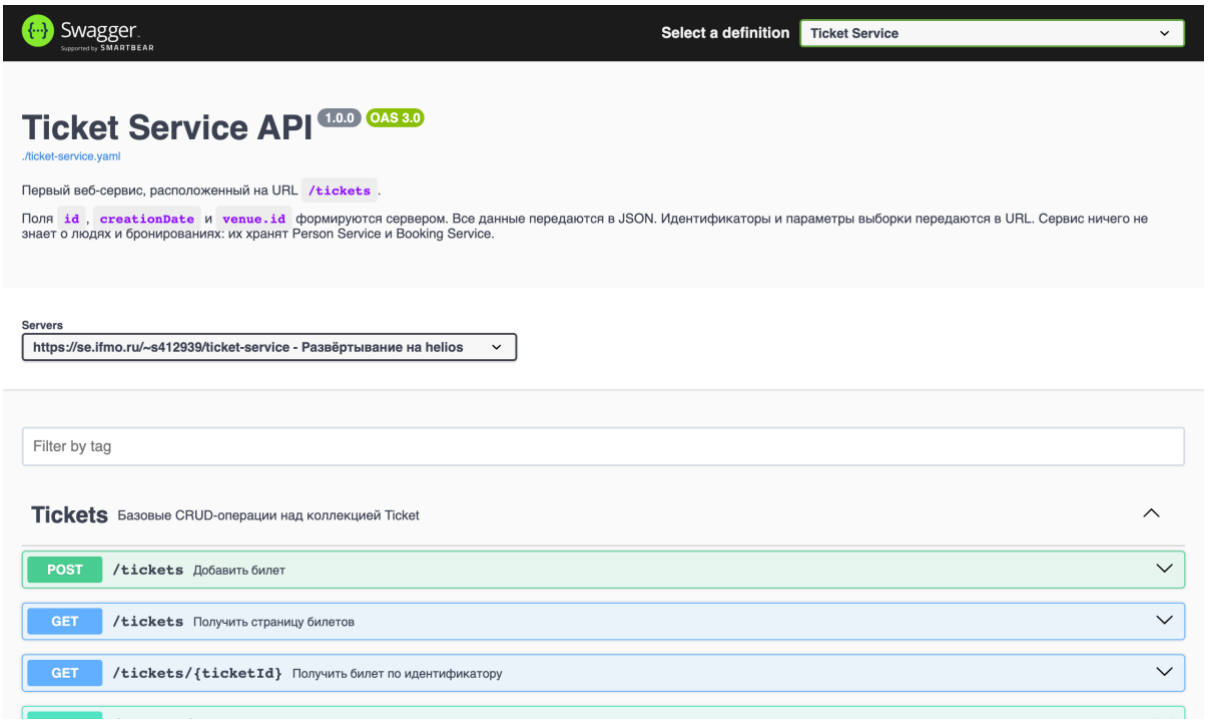


Рис 5. Swagger UI для Ticket Service

Выводы

Реализованы три веб-сервиса на Spring MVC REST и клиентское приложение на React. Мы научились генерировать контроллеры, модели и REST-клиенты из спецификации OpenAPI, реализовывать фильтрацию, сортировку и постраничную выборку, возвращать ошибки валидации в едином формате, разворачивать сервисы в отдельных экземплярах Tomcat с доступом только по https и самоподписанным сертификатом.